Searching Algorithms and Complexity

Linear search, binary search, and why sorted data matters

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 19 Tuesday, November 10, 2026

Today

1. The **guessing game**: how fast can you find a number?
2. **Linear search**: look at everything. Cost $O(n)$.
3. **Binary search**: throw away half each time. Cost $O(\log n)$. It needs **sorted** data.
4. A trace with `lo`, `hi`, `mid`, and the bugs that usually hide there.
5. When is it worth **sorting first**? A first look at comparing algorithms.

- **Exercise 8 (Shunting-Yard Trace)** is due **Thursday, Nov 12**.
- Next class starts sorting. Today's ideas on $O(\log n)$ and "halving" return there.

Question: Guess My Number

I am thinking of a whole number from **1 to 100**. After each guess I say **higher** or **lower**.

(a) What is a good first guess? Why?
(b) What is the **largest** number of guesses you could ever need with your strategy?

Question: Guess My Number

I am thinking of a whole number from **1 to 100**. After each guess I say **higher** or **lower**.

(a) What is a good first guess? Why?
(b) What is the **largest** number of guesses you could ever need with your strategy?

(a) 50, the middle: whatever I say, you remove **half** of the possibilities.

(b) 7. Each guess halves the range:
$100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$. In general you need $\lfloor \log_2 n \rfloor + 1$ guesses: $2^7 = 128 \geq 100$, while six guesses can tell apart at most $2^6 = 64 < 100$ numbers.

Analogy: looking up a word in a paper dictionary. You open it near the middle and decide "before or after?" You never read it page by page. That only works because the words are **in order**.

Linear Search

Look at the elements one at a time, from the front.

```
int linearSearch(const std::vector<int>& a, int target) {
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        if (a[i] == target) { return i; } // found: stop at once
    }
    return -1;                                  // not found
}
```

Question. With `a = {7, 3, 9, 4}`: how many comparisons does `linearSearch(a, 9)` make? And `linearSearch(a, 5)`?

Linear Search

Look at the elements one at a time, from the front.

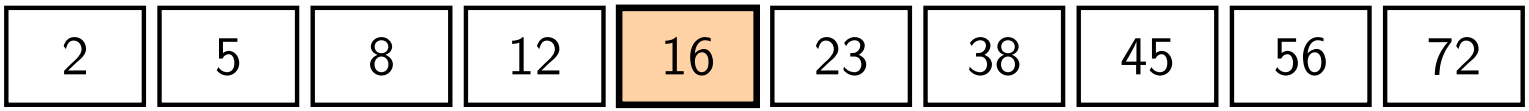
```
int linearSearch(const std::vector<int>& a, int target) {
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        if (a[i] == target) { return i; } // found: stop at once
    }
    return -1;                                  // not found
}
```

Question. With `a = {7, 3, 9, 4}`: how many comparisons does `linearSearch(a, 9)` make? And `linearSearch(a, 5)`?

9: 3 comparisons (it returns index 2). **5:** all 4, and it returns -1. In the worst case (absent, or last) it makes n comparisons: $O(n)$. On average, about $(n+1)/2$ comparisons, if the target is equally likely to be at any position. **It works on any data**: sorted or not, vector or linked list.

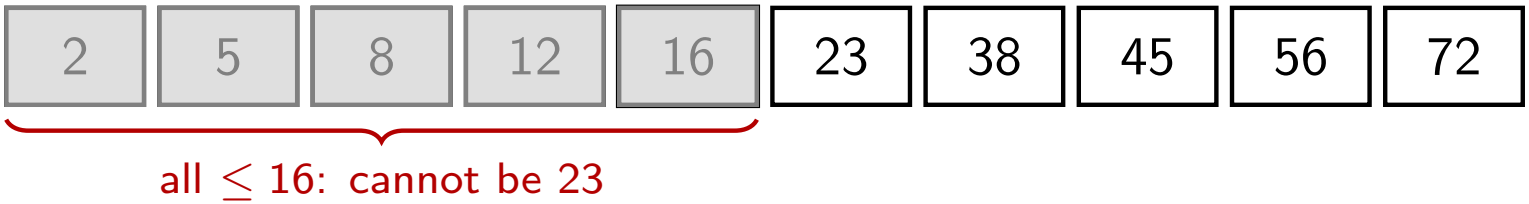
Why Sorted Data Helps

Question. In the sorted array below we look at the middle element, 16, while searching for 23. What do we learn about the elements to the **left** of 16?



Why Sorted Data Helps

Question. In the sorted array below we look at the middle element, 16, while searching for 23. What do we learn about the elements to the **left** of 16?



All of them are $\leq 16 < 23$. One comparison eliminates 16 and everything to its left: about **half** of the array. In an **unsorted** array, the middle element tells us **nothing** about the other elements: we cannot skip anything.

Binary Search

Keep a range `lo..hi` that **must contain the target if it is present**. Compare with the middle, and keep the correct half.

```
int binarySearch(const std::vector<int>& a, int target) {
    int lo = 0, hi = static_cast<int>(a.size()) - 1; // target, if present,
         is in a[lo..hi]
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { return mid; }
        if (a[mid] < target) { lo = mid + 1; }     // discard the left half
             and mid
        else                 { hi = mid - 1; }     // discard the right half
             and mid
    }
    return -1;                                     // empty range: not found
}
```

- `mid = lo + (hi - lo) / 2` instead of `(lo + hi) / 2`: the sum `lo + hi` can overflow `int` for huge arrays; the subtraction cannot.
- The $+1$ **and** -1 matter: `mid` has been checked, so it must leave the range.

Trace: Searching for 23

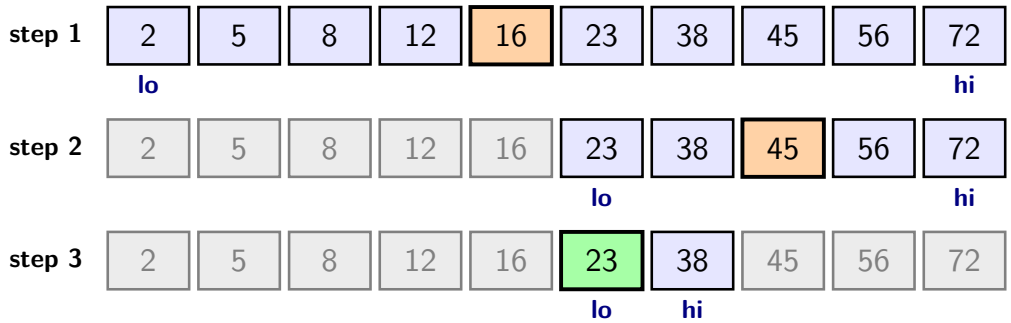
`a = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` (10 elements), target **23**. Predict `mid` and the decision, then check.

step	`lo`	`hi`	`mid`	`a[mid]`	decision
1	0	9	4	16	
2	5	9	7	45	
3	5	6	5	23	

Trace: Searching for 23

`a = {2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` (10 elements), target **23**. Predict `mid` and the decision, then check.

step	`lo`	`hi`	`mid`	`a[mid]`	decision
1	0	9	4	16	$a[mid] <$ target: `lo = mid + 1`
2	5	9	7	45	$a[mid] >$ target: `hi = mid - 1`
3	5	6	5	23	**found**: return 5



Trace: Searching for 40 (Not Present)

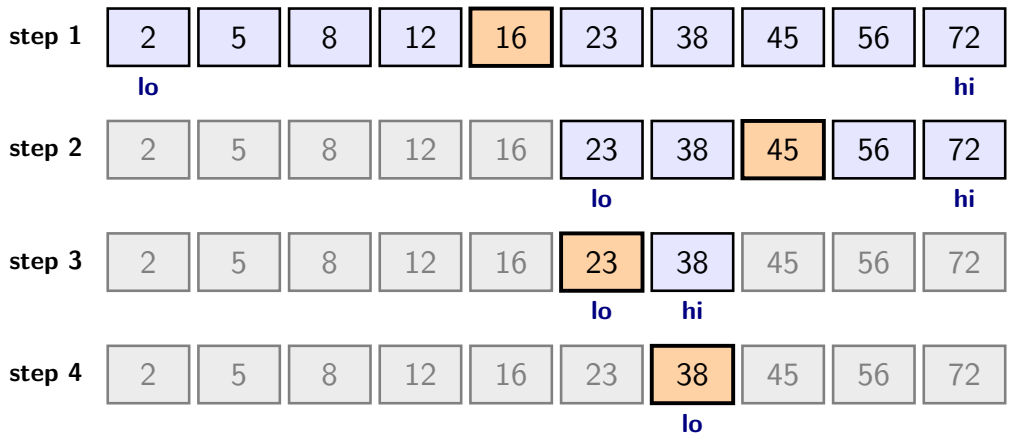
Same array, target **40**. What happens when the target is absent?

step	`lo`	`hi`	`mid`	`a[mid]`	decision
1	0	9	4	16	
2	5	9	7	45	
3	5	6	5	23	
4	6	6	6	38	

Trace: Searching for 40 (Not Present)

Same array, target **40**. What happens when the target is absent?

step	lo	hi	mid	a[mid]	decision
1	0	9	4	16	$a[mid] <$ target: `lo = mid + 1`
2	5	9	7	45	$a[mid] >$ target: `hi = mid - 1`
3	5	6	5	23	$a[mid] <$ target: `lo = mid + 1`
4	6	6	6	38	$a[mid] <$ target: `lo = mid + 1`



After step 4, `lo = 7` and `hi = 6`: the range is **empty**, the loop ends, and the function returns -1. Four comparisons for 10 elements.

Question: How Many Comparisons?

(a) Binary search on $n = 1\,000\,000$ sorted numbers: about how many comparisons, in the worst case?
(b) What if the array doubles to $2\,000\,000$?

n	**Linear (worst)**	**Binary (worst)**
10	10	
100	100	
1 000	1 000	
1 000 000	1 000 000	
1 000 000 000	1 000 000 000	

Question: How Many Comparisons?

(a) Binary search on $n = 1\,000\,000$ sorted numbers: about how many comparisons, in the worst case?
(b) What if the array doubles to $2\,000\,000$?

n	**Linear (worst)**	**Binary (worst)**
10	10	4
100	100	7
1 000	1 000	10
1 000 000	1 000 000	20
1 000 000 000	1 000 000 000	30

(a) 20. **(b)** 21: doubling the array adds **one** worst-case comparison. Worst case is $\lfloor \log_2 n \rfloor + 1$ (I checked this by running every possible search for these sizes).

Why Is It $O(\log n)$?

Each step discards at least half of the remaining range. The range sizes are

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \quad \ldots, \quad 1$$

Question: how many times can you halve n before you reach 1?

Why Is It $O(\log n)$?

Each step discards at least half of the remaining range. The range sizes are

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \quad \ldots, \quad 1$$

Question: how many times can you halve n before you reach 1?

About $\log_2 n$ times, because $n/2^k \leq 1$ exactly when $k \geq \log_2 n$. So there are about $\log_2 n$ loop iterations (at most $\lfloor \log_2 n \rfloor + 1$, as the table showed), and each iteration does a constant amount of work. So the total is $O(\log n)$.

Recurrence form: $T(n) = T(n/2) + O(1)$. (Next week's merge sort has $T(n) = 2T(n/2) + O(n)$.)

Rule of thumb: a loop that halves (or doubles) something each time, with $O(1)$ work per pass, is $O(\log n)$.

Question: A Bug Hiding in the Update

A student writes `lo = mid;` (no `+ 1`) in the first branch. Run it on `a = {1, 2}`, target **2**. `lo = 0, hi = 1`.

(a) What is `mid`? What does the code do next?
(b) What happens on the next iteration?

Question: A Bug Hiding in the Update

A student writes `lo = mid;` (no `+ 1`) in the first branch. Run it on `a = {1, 2}`, target **2**. `lo = 0, hi = 1`.

(a) What is `mid`? What does the code do next?
(b) What happens on the next iteration?

(a) `mid = 0 + (1 - 0)/2 = 0`. `a[0] = 1 < 2`, so `lo = mid = 0`: **nothing changed**.
(b) The same state repeats forever: an **infinite loop**. The range never shrinks because `mid` was not removed. (I ran it: after 6 iterations, `lo = 0, hi = 1, mid = 0` every time.)

Rule: every step must **shrink the range**. `mid` has just been tested, so drop it: `mid + 1` or `mid - 1`.

Does the Data Structure Matter?

Binary search needs to jump to the middle **in one step**.

Question: you have a sorted **linked list** of n numbers. Can you do binary search on it in $O(\log n)$ time?

Does the Data Structure Matter?

Binary search needs to jump to the middle **in one step**.

Question: you have a sorted **linked list** of n numbers. Can you do binary search on it in $O(\log n)$ time?

No. A linked list has no `a[mid]`: reaching the middle node means following $n/2$ `next` pointers, which is $O(n)$ work. In a straightforward implementation the successive midpoints cost about $n/2 + n/4 + \cdots = O(n)$ hops in total: only $O(\log n)$ comparisons, but the pointer-following ruins the $O(\log n)$ time.

Needs random access: an array or a `std::vector`, where `a[i]` is $O(1)$ (like your `MiniVector`).

Two requirements: (1) the data is **sorted**; (2) we can reach any element in $O(1)$.

Question: Sort First?

A `vector` holds $n = 1\,000\,000$ numbers in **random order**. Sorting costs about $n \log_2 n \approx 20\,000\,000$ steps (Class 20 will show this).

(a) You will search for **one** number. Sort first, or just do a linear search?
(b) You will search for **1 000 different** numbers. Sort first?

Question: Sort First?

A `vector` holds $n = 1\,000\,000$ numbers in **random order**. Sorting costs about $n \log_2 n \approx 20\,000\,000$ steps (Class 20 will show this).

(a) You will search for **one** number. Sort first, or just do a linear search?
(b) You will search for **1 000 different** numbers. Sort first?

(a) Linear search: $\approx 10^6$ steps, which is less than the 2×10^7 steps to sort.

(b) 1 000 linear searches: $1000 \times 10^6 = 10^9$ steps. Sort once (2×10^7), then 1 000 binary searches ($1000 \times 20 = 2 \times 10^4$): about 2×10^7 in total. **Sorting wins by a factor of 50.**

Lesson: sorting is an investment. It pays off when you search many times. Complexity helps you decide. (These are approximate step counts, not measured running times.)

Growth Rates We Have Met

Illustrative step counts for $n = 1\,000\,000$ (constant factors ignored):

Growth	**Example**	**Steps**
$O(1)$	`v[i]`, push to a stack	1
$O(\log n)$	binary search	20
$O(n)$	linear search, traversing a list	$1\,000\,000$
$O(n \log n)$	sorting well (Class 20–21)	$\approx 20\,000\,000$
$O(n^2)$	nested loops, simple sorts (next class)	10^{12}

Growth Rates We Have Met

Illustrative step counts for $n = 1\,000\,000$ (constant factors ignored):

Growth	**Example**	**Steps**
$O(1)$	`v[i]`, push to a stack	1
$O(\log n)$	binary search	20
$O(n)$	linear search, traversing a list	$1\,000\,000$
$O(n \log n)$	sorting well (Class 20–21)	$\approx 20\,000\,000$
$O(n^2)$	nested loops, simple sorts (next class)	10^{12}

Reading the table: $\log n$ grows very slowly, $n \log n$ is fine, n^2 is already too slow for a million items. At 10^9 simple operations per second, 10^{12} steps takes about 17 minutes; 2×10^7 steps takes 0.02 seconds.

Common Mistakes

- **Binary search on unsorted data**: the answer is wrong, and nothing warns you.
- **Updating with `lo = mid` or `hi = mid` in the `lo <= hi` version**: the loop can run forever.
- **Using `lo < hi` where `lo <= hi` is needed**: misses the case of one element left.
- **`(lo + hi) / 2` for huge arrays**: the sum can overflow. Use `lo + (hi - lo) / 2`.
- **Expecting the first duplicate**: plain binary search returns *some* match, not necessarily the first. (Optional supplement: `lowerBound`.)
- **Calling a loop that halves "$O(n)$" or a nested loop "$O(\log n)$"**: count how the range shrinks.

Practice

1. On `{2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` search for **72**. List `mid` at each step.
2. How many comparisons are needed (worst case) for $n = 1\,000$?
3. Why must the array be sorted *in the same order* as the comparison (ascending for <)?
4. A search is $O(\log n)$. If n grows from 10^6 to 10^9, how many more steps?
5. Binary search on a linked list: why is it $O(n)$, not $O(\log n)$?

Practice

1. On `{2, 5, 8, 12, 16, 23, 38, 45, 56, 72}` search for **72**. List `mid` at each step.
2. How many comparisons are needed (worst case) for $n = 1\,000$?
3. Why must the array be sorted *in the same order* as the comparison (ascending for <)?
4. A search is $O(\log n)$. If n grows from 10^6 to 10^9, how many more steps?
5. Binary search on a linked list: why is it $O(n)$, not $O(\log n)$?

1. `lo, hi = 0, 9`: mid 4 (16); `lo = 5`: mid 7 (45); `lo = 8`: mid 8 (56); `lo = 9`: mid 9 (72), found. Four steps.
2. 10.
3. The decision "go left if `a[mid] > target`" assumes the left side holds smaller values. On descending data the rule must be reversed.
4. From 20 to 30: only 10 more steps for 1000 times more data.
5. Reaching the middle node needs $n/2$ pointer hops.

Summary and Preview

- **Linear search:** any data, $O(n)$. **Binary search:** sorted data with random access, $O(\log n)$.
- Binary search keeps the invariant "the target, if present, is in `lo..hi`" and shrinks the range every step.
- **Doubling the data adds one comparison.**
- **Sorting is an investment**: it pays off after many searches.

Coming up

Nov 12: Sorting I: bubble, selection, and insertion sort, and why they are $O(n^2)$. Exercise 8 due; Exercise 9 (Sorting Trace) posted.
Nov 17: merge sort, using the same halving idea.